Practical-1

Aim

To implement different sorting algorithms and analyze their time complexities.

Theory

Sorting is the process of arranging elements in a particular order, usually **ascending or descending order**.

The sorting algorithms used are:

- **Bubble Sort:** Repeatedly compares adjacent elements and swaps them if they are in the wrong order.
- **Selection Sort:** Finds the minimum element and places it at the correct position.
- **Insertion Sort:** Inserts each element into its proper position in the sorted part.
- **Merge Sort:** Divides the array into smaller parts, sorts them and merges them.
- **Quick Sort:** Selects a pivot and partitions the array around the pivot.

Time Complexity:

Algorithm	Best Case	Average Case	Worst Case
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$
Merge Sort	O(n log n)	O(n log n)	O(n log n)
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$

Algorithm / Pseudocode

1. Bubble Sort

```
None
BubbleSort(A, n)
   for i = 0 to n-2
      for j = 0 to n-i-2
```

Name: V V V L Sravya (92460118112)

```
        if A[j] > A[j+1]
            swap A[j], A[j+1]
```

2. Selection Sort

```
None
SelectionSort(A, n)
    for i = 0 to n-2
        min = i
        for j = i+1 to n-1
            if A[j] < A[min]
                min = j
        swap A[i], A[min]
```

3. Insertion Sort

```
None
InsertionSort(A, n)
    for i = 1 to n-1
        key = A[i]
        j = i-1
        while j >= 0 and A[j] > key
              A[j+1] = A[j]
            j = j-1
         A[j+1] = key
```

4. Merge Sort

```
None
MergeSort(A, low, high)
    if low < high
        mid = (low + high) / 2
        MergeSort(A, low, mid)
        MergeSort(A, mid+1, high)
        Merge(A, low, mid, high)
```

5. Quick Sort

```
None
QuickSort(A, low, high)
   if low < high
      p = Partition(A, low, high)
      QuickSort(A, low, p-1)
      QuickSort(A, p+1, high)
```

Program

```
#include <iostream>
using namespace std;
// Bubble Sort
void bubbleSort(int a[], int n) {
   for (int i = 0; i < n - 1; i++) {
      for (int j = 0; j < n - i - 1; j++) {
         if (a[j] > a[j + 1])
            swap(a[j], a[j + 1]);
      }
   }
}

// Selection Sort
void selectionSort(int a[], int n) {
   for (int i = 0; i < n - 1; i++) {
      int min = i;
      for (int j = i + 1; j < n; j++) {
         if (a[j] < a[min])
            min = j;
      }
      swap(a[i], a[min]);
   }
}

// Insertion Sort
void insertionSort(int a[], int n) {
   for (int i = 1; i < n; i++) {
      int key = a[i];
      int j = i - 1;
      while (j >= 0 && a[j] > key) {
         a[j + 1] = a[j];
         j--;
      }
```

```
        a[j + 1] = key;
    }
}

// Merge Sort
void merge(int a[], int low, int mid, int high) {
    int temp[100], i = low, j = mid + 1, k = low;
    while (i <= mid && j <= high) {
        if (a[i] < a[j])
            temp[k++] = a[i++];
        else
            temp[k++] = a[j++];
    }
    while (i <= mid)
        temp[k++] = a[i++];
    while (j <= high)
        temp[k++] = a[j++];
    for (i = low; i <= high; i++)
        a[i] = temp[i];
}
void mergeSort(int a[], int low, int high) {
    if (low < high) {
        int mid = (low + high) / 2;
        mergeSort(a, low, mid);
        mergeSort(a, mid + 1, high);
        merge(a, low, mid, high);
    }
}

// Quick Sort
int partition(int a[], int low, int high) {
    int pivot = a[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (a[j] < pivot) {
            i++;
            swap(a[i], a[j]);
        }
    }
    swap(a[i + 1], a[high]);
    return i + 1;
}
void quickSort(int a[], int low, int high) {
    if (low < high) {
```

```
        int p = partition(a, low, high);
        quickSort(a, low, p - 1);
        quickSort(a, p + 1, high);
    }
}
// Display array
void display(int a[], int n) {
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";
    cout << endl;
}
int main() {
    int a[100], n, choice;
    cout << "Enter number of elements: ";
    cin >> n;
    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> a[i];
    cout << "\n1. Bubble Sort";
    cout << "\n2. Selection Sort";
    cout << "\n3. Insertion Sort";
    cout << "\n4. Merge Sort";
    cout << "\n5. Quick Sort";
    cout << "\nEnter your choice: ";
    cin >> choice;
    switch (choice) {
        case 1:
            bubbleSort(a, n);
            break;
        case 2:
            selectionSort(a, n);
            break;
        case 3:
            insertionSort(a, n);
            break;
        case 4:
            mergeSort(a, 0, n - 1);
            break;
        case 5:
            quickSort(a, 0, n - 1);
            break;
        default:
            cout << "Invalid choice";
            return 0;
```

```
    }
    cout << "\nSorted array: ";
    display(a, n);

    return 0;
}
```

```
vvvlsravya@Vs-MacBook-Air-377 Practical_1-10 % g++ practical1_sorting.cpp -o practical1 && ./practical1
Enter number of elements: 6
Enter elements: 19
89
24
68
22
95

1. Bubble Sort
2. Selection Sort
3. Insertion Sort
4. Merge Sort
5. Quick Sort
Enter your choice: 1

Sorted array: 19 22 24 68 89 95
vvvlsravya@Vs-MacBook-Air-377 Practical_1-10 % g++ practical1_sorting.cpp -o practical1 && ./practical1
Enter number of elements: 5
Enter elements: 19
78
95
22
18

1. Bubble Sort
2. Selection Sort
3. Insertion Sort
4. Merge Sort
5. Quick Sort
Enter your choice: 2

Sorted array: 18 19 22 78 95
vvvlsravya@Vs-MacBook-Air-377 Practical_1-10 %
```

Result

Thus, the given sorting algorithms were implemented successfully and their time complexities were analyzed.

PRACTICAL-2

Aim

To implement **Linear Search and Binary Search algorithms** and analyze their time complexities.

Theory

Linear Search checks each element one by one until the required element is found or the array ends.

Binary Search searches an element in a **sorted array** by repeatedly dividing the search range into two halves.

Time Complexity

Algorithm	Best Case	Average Case	Worst Case
Linear Search	O(1)	O(n)	O(n)
Binary Search	O(1)	O(log n)	O(log n)

Algorithm / Pseudocode

1. Linear Search

```
None
LinearSearch(A, n, key)
   for i = 0 to n-1
      if A[i] == key
         return i
   return -1
```

2. Binary Search

```
None
BinarySearch(A, n, key)
   low = 0
   high = n-1
   while low <= high
```

Name: V V V L Sravya (92460118112)

```
    mid = (low + high) / 2
    if A[mid] == key
        return mid
    else if A[mid] < key
        low = mid + 1
    else
        high = mid - 1
  return -1
```

Program

```
#include <iostream>
using namespace std;
// Linear Search
int linearSearch(int a[], int n, int key) {
    for (int i = 0; i < n; i++) {
        if (a[i] == key)
            return i;
    }
    return -1;
}
// Binary Search
int binarySearch(int a[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (a[mid] == key)
            return mid;
        else if (a[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}
int main() {
    int a[100], n, key, choice, result;
    cout << "Enter number of elements: ";
    cin >> n;
    cout << "Enter elements in sorted order: ";
    for (int i = 0; i < n; i++)
        cin >> a[i];
```

```
    cout << "Enter element to search: ";
    cin >> key;
    cout << "\n1. Linear Search";
    cout << "\n2. Binary Search";
    cout << "\nEnter your choice: ";
    cin >> choice;
    if (choice == 1)
        result = linearSearch(a, n, key);
    else if (choice == 2)
        result = binarySearch(a, n, key);
    else {
        cout << "Invalid choice";
        return 0;
    }
    if (result != -1)
        cout << "Element found at position " << result + 1 << endl;
    else
        cout << "Element not found" << endl;

    return 0;
}
```

```
vvvlsravya@Vs-MacBook-Air-377 Practical_1-10 % g++ practical2_search.cpp -o practical2 && ./practical2
Enter number of elements: 7
Enter elements in sorted order: 12
28
39
44
49
57
69
Enter element to search: 12

1. Linear Search
2. Binary Search
Enter your choice: 2
Element found at position 1
vvvlsravya@Vs-MacBook-Air-377 Practical_1-10 % g++ practical2_search.cpp -o practical2 && ./practical2
Enter number of elements: 5
Enter elements in sorted order: 39
27
96
12
98
Enter element to search: 98

1. Linear Search
2. Binary Search
Enter your choice: 1
Element found at position 5
vvvlsravya@Vs-MacBook-Air-377 Practical_1-10 %
```

Result

Thus, **Linear Search and Binary Search** were implemented successfully and their time complexities were analyzed.

PRACTICAL-3

Aim

To implement the **Max-Heap Sort algorithm** and sort the given elements in ascending order.

Theory

A **Max-Heap** is a complete binary tree in which the value of every parent node is greater than or equal to its children.

In Heap Sort, we first build a Max-Heap. Then the largest element (root) is swapped with the last element and the heap size is reduced. This process is repeated until the array is sorted.

Time Complexity

Operation	Time Complexity
Building Max-Heap	O(n)
Heap Sort	O(n log n)
Best Case	O(n log n)
Average Case	O(n log n)
Worst Case	O(n log n)
Space Complexity	O(1)

Name: V V V L Sravya (92460118112)

Algorithm / Pseudocode

Max-Heapify

```
None
MaxHeapify(A, n, i)
    largest = i
    left = 2*i + 1
    right = 2*i + 2
    if left < n and A[left] > A[largest]
        largest = left
    if right < n and A[right] > A[largest]
        largest = right
    if largest != i
        swap A[i], A[largest]
        MaxHeapify(A, n, largest)
```

Heap Sort

```
None
HeapSort(A, n)
    for i = n/2 - 1 down to 0
        MaxHeapify(A, n, i)
    for i = n-1 down to 1
        swap A[0], A[i]
        MaxHeapify(A, i, 0)
```

Program:

```
#include <iostream>
using namespace std;
void maxHeapify(int a[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    if (left < n && a[left] > a[largest])
        largest = left;
    if (right < n && a[right] > a[largest])
        largest = right;
    if (largest != i) {
        swap(a[i], a[largest]);
        maxHeapify(a, n, largest);
    }
}
void heapSort(int a[], int n) {
    // Build Max-Heap
    for (int i = n / 2 - 1; i >= 0; i--)
        maxHeapify(a, n, i);
    // Extract elements from heap
    for (int i = n - 1; i > 0; i--) {
        swap(a[0], a[i]);
```

```
        maxHeapify(a, i, 0);
    }
}
int main() {
    int a[100], n;
    cout << "Enter number of elements: ";
    cin >> n;
    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> a[i];
    heapSort(a, n);
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";
    return 0;
}
```

```
vvvlsravya@Vs-MacBook-Air-377 Practical_1-10 % g++ practical3_heap.cpp -o practical3 && ./practical3
Enter number of elements: 7
Enter elements: 92
79
58
99
87
27
19
Sorted array: 19 27 58 79 87 92 99 %
vvvlsravya@Vs-MacBook-Air-377 Practical_1-10 %
```

Result

Thus, the **Max-Heap Sort algorithm** was implemented successfully and the elements were sorted in ascending order.

PRACTICAL-4

Aim

To implement a **factorial program using iterative and recursive methods** and analyze their time complexities.

Theory

The **factorial** of a non-negative integer n is the product of all positive integers from 1 to n.

$n!=n\times(n-1)\times\cdots\times1$

For example:

```
5! = 5 × 4 × 3 × 2 × 1 = 120
```

- **Iterative method:** Uses a loop to calculate the factorial.
- **Recursive method:** A function calls itself until the base condition is reached.

Time Complexity

Method	Time Complexity	Space Complexity
Iterative	O(n)	O(1)
Recursive	O(n)	O(n)

Algorithm / Pseudocode

1. Iterative Factorial

```
None
Factorial_Iterative(n)
    fact = 1
```

Name: V V V L Sravya (92460118112)

```
    for i = 1 to n
        fact = fact * i
    return fact
```

2. Recursive Factorial

```
None
Factorial_Recursive(n)
    if n == 0 or n == 1
        return 1
    return n * Factorial_Recursive(n-1)
```

Program

```
#include <iostream>
using namespace std;
// Iterative method
long long factorialIterative(int n) {
    long long fact = 1;
    for (int i = 1; i <= n; i++)
        fact = fact * i;
    return fact;
}
// Recursive method
long long factorialRecursive(int n) {
    if (n == 0 || n == 1)
```

```
        return 1;
    return n * factorialRecursive(n - 1);
}
int main() {
    int n;
    cout << "Enter a number: ";
    cin >> n;
    cout << "Factorial using Iterative Method: "
         << factorialIterative(n) << endl;
    cout << "Factorial using Recursive Method: "
         << factorialRecursive(n) << endl;
    return 0;
}
```

```
vvvlsravya@Vs-MacBook-Air-377 Practical_1-10 % g++ practical4_factorial.cpp -o practical4 && ./practical4
Enter a number: 6
Factorial using Iterative Method: 720
Factorial using Recursive Method: 720
vvvlsravya@Vs-MacBook-Air-377 Practical_1-10 % g++ practical4_factorial.cpp -o practical4 && ./practical4
Enter a number: 78
Factorial using Iterative Method: 0
Factorial using Recursive Method: 0
vvvlsravya@Vs-MacBook-Air-377 Practical_1-10 % g++ practical4_factorial.cpp -o practical4 && ./practical4
Enter a number: 8
Factorial using Iterative Method: 40320
Factorial using Recursive Method: 40320
vvvlsravya@Vs-MacBook-Air-377 Practical_1-10 %
```

Result

Thus, the factorial of a given number was calculated successfully using **iterative and recursive methods**, and their time complexities were analyzed.

PRACTICAL-5

Aim

To implement the **0/1 Knapsack problem using Dynamic Programming**.

Theory

The **0/1 Knapsack problem** is used to select items with given weights and profits to maximize the total profit without exceeding the given knapsack capacity.

Each item can either be **selected (1)** or **not selected (0)**.

Dynamic Programming stores solutions to smaller subproblems in a table to avoid repeated calculations.

Time Complexity

- **Time Complexity:** O(n × W)
- **Space Complexity:** O(n × W)

Where:

- `n` = number of items
- `W` = capacity of the knapsack

Algorithm / Pseudocode

```
None

Knapsack(weight, profit, n, W)
   Create table K[n+1][W+1]
   for i = 0 to n
      for w = 0 to W
         if i == 0 or w == 0
            K[i][w] = 0
         else if weight[i-1] <= w
```

```
            K[i][w] = max(
                profit[i-1] + K[i-1][w-weight[i-1]],
                K[i-1][w]
            )
        else
            K[i][w] = K[i-1][w]
    return K[n][W]
```

Program

```
#include <iostream>
#include <algorithm>
using namespace std;
int main() {
    int n, W;
    cout << "Enter number of items: ";
    cin >> n;
    int weight[100], profit[100];
    cout << "Enter weights: ";
    for (int i = 0; i < n; i++)
        cin >> weight[i];
    cout << "Enter profits: ";
    for (int i = 0; i < n; i++)
        cin >> profit[i];
```

```
    cout << "Enter knapsack capacity: ";
    cin >> W;
    int K[101][101];
    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {

            if (i == 0 || w == 0)
                K[i][w] = 0;
            else if (weight[i - 1] <= w)
                K[i][w] = max(
                    profit[i - 1] + K[i - 1][w - weight[i - 1]],
                    K[i - 1][w]
                );
            else
                K[i][w] = K[i - 1][w];
        }
    }
    cout << "Maximum Profit = " << K[n][W] << endl;
    return 0;
}
```

```
vvvlsravya@Vs-MacBook-Air-377 Practical_1-10 % g++ practical5_knapsack.cpp -o practical5 && ./practical5
Enter number of items: 8
Enter weights: 2
7
9
1
3
4
8
5
Enter profits: 22
17
18
12
7
19
21
3
Enter knapsack capacity: 9
Maximum Profit = 53
vvvlsravya@Vs-MacBook-Air-377 Practical_1-10 %
```

Result

Thus, the **0/1 Knapsack problem** was successfully implemented using **Dynamic Programming**, and the maximum profit was obtained.

PRACTICAL-6

Aim

To implement **Matrix Chain Multiplication using Dynamic Programming** and find the minimum number of scalar multiplications required.

Theory

Matrix Chain Multiplication determines the **best order to multiply a sequence of matrices** so that the total number of scalar multiplications is minimum.

The order of matrices cannot be changed, but the **parentheses can be placed differently**.

For matrices A of size p × q and B of size q × r, their multiplication requires:

p × q × r scalar multiplications.

Algorithm / Pseudocode

```
None

MatrixChain(p, n)
    Create table m[n][n]
    for i = 1 to n
        m[i][i] = 0
    for length = 2 to n
        for i = 1 to n-length+1
            j = i + length - 1
            m[i][j] = infinity
            for k = i to j-1
                cost = m[i][k] + m[k+1][j]
                    + p[i-1] * p[k] * p[j]
```

Name: V V V L Sravya (92460118112)

```
            if cost < m[i][j]
                m[i][j] = cost
    return m[1][n]
```

Program

```
#include <iostream>
#include <climits>
using namespace std;
int main() {
    int n;
    cout << "Enter number of matrices: ";
    cin >> n;
    int p[100];
    cout << "Enter dimensions of matrices: ";
    cout << "\nFor n matrices, enter n+1 dimensions: ";
    for (int i = 0; i <= n; i++)
        cin >> p[i];
    int m[100][100];
    // Cost of multiplying one matrix is zero
    for (int i = 1; i <= n; i++)
        m[i][i] = 0;
    // Chain length
    for (int length = 2; length <= n; length++) {
```

```
        for (int i = 1; i <= n - length + 1; i++) {
            int j = i + length - 1;
            m[i][j] = INT_MAX;
            for (int k = i; k < j; k++) {
                int cost = m[i][k]
                        + m[k + 1][j]
                        + p[i - 1] * p[k] * p[j];
                if (cost < m[i][j])
                    m[i][j] = cost;
            }
        }
    }
    cout << "Minimum number of scalar multiplications = "
         << m[1][n] << endl;
    return 0;
}
```

```
vvvlsravya@Vs-MacBook-Air-377 Practical_1-10 % g++ practical6_matrix_chain.cpp -o practical6 && ./practical6
Enter number of matrices: 4
Enter dimensions of matrices:
For n matrices, enter n+1 dimensions: 5
3
3
5
6
Minimum number of scalar multiplications = 225
vvvlsravya@Vs-MacBook-Air-377 Practical_1-10 %
```

Result

Thus, **Matrix Chain Multiplication** was successfully implemented using **Dynamic Programming**, and the minimum number of scalar multiplications was obtained.

PRACTICAL-7

Aim

To implement the **Making Change problem using Dynamic Programming** and find the minimum number of coins required to make a given amount.

Theory

The **Making Change problem** finds the minimum number of coins needed to make a given amount using a set of coin denominations.

Dynamic Programming stores the minimum number of coins required for each smaller amount and uses these results to solve the larger amount.

Where:

- `n` = number of coin denominations
- `A` = amount to be made

Algorithm / Pseudocode

```
None

MakingChange(coins, n, amount)

  Create array dp[amount + 1]

  dp[0] = 0

  for i = 1 to amount

    dp[i] = infinity


  for i = 1 to amount

    for j = 0 to n-1


      if coins[j] <= i
```

```
            dp[i] = min(dp[i],
                        1 + dp[i - coins[j]])

    return dp[amount]
```

Program

```
#include <iostream>
#include <algorithm>
#include <climits>
using namespace std;
int main() {
    int n, amount;
    cout << "Enter number of coin denominations: ";
    cin >> n;
    int coins[100];
    cout << "Enter coin denominations: ";
    for (int i = 0; i < n; i++)
        cin >> coins[i];
    cout << "Enter amount: ";
    cin >> amount;
    int dp[1000];
    dp[0] = 0;
    for (int i = 1; i <= amount; i++)
```

```
        dp[i] = INT_MAX;
    // Calculate minimum coins
    for (int i = 1; i <= amount; i++) {
        for (int j = 0; j < n; j++) {
            if (coins[j] <= i && dp[i - coins[j]] != INT_MAX) {
                dp[i] = min(dp[i],
                            1 + dp[i - coins[j]]);
            }
        }
    }
    if (dp[amount] == INT_MAX)
        cout << "Amount cannot be formed using given coins.";
    else
        cout << "Minimum number of coins = "
             << dp[amount] << endl;
    return 0;
}
```

```
vvvlsravya@Vs-MacBook-Air-377 Practical_1-10 % g++ practical7_making_change.cpp -o practical7 && ./practical7
Enter number of coin denominations: 6
Enter coin denominations: 2
7
5
3
1
8
Enter amount: 90
Minimum number of coins = 12
vvvlsravya@Vs-MacBook-Air-377 Practical_1-10 %
```

Result

Thus, the **Making Change problem** was successfully implemented using **Dynamic Programming**, and the minimum number of coins required was obtained.

PRACTICAL-8

Aim

To implement a **Graph** and perform **Depth First Search (DFS)** and **Breadth First Search (BFS)**.

Theory

A **graph** consists of a set of **vertices (nodes)** and **edges** connecting them.

- **DFS:** Visits a vertex and then explores as far as possible along each branch before backtracking. It uses a **stack or recursion**.
- **BFS:** Visits all adjacent vertices first and then moves to the next level. It uses a **queue**.

Time Complexity

For both DFS and BFS:

- **Time Complexity:** O(V + E)
- **Space Complexity:** O(V)

Where:

- V = number of vertices
- E = number of edges

Algorithm / Pseudocode

1. DFS

```
None
DFS(v)
    Mark v as visited
    Print v
    for each adjacent vertex u of v
        if u is not visited
            DFS(u)
```

Name: V V V L Sravya (92460118112)

2. BFS

```
None
BFS(start)
    Create an empty queue
   Mark start as visited
   Insert start into queue
   while queue is not empty
      v = remove front vertex
      Print v
      for each adjacent vertex u of v
         if u is not visited
            Mark u as visited
            Insert u into queue
```

Program

```
#include <iostream>
#include <queue>
using namespace std;
int graph[100][100];
bool visited[100];
int n;
// DFS
void DFS(int v) {
   visited[v] = true;
```

```
    cout << v << " ";
    for (int i = 0; i < n; i++) {
        if (graph[v][i] == 1 && !visited[i])
            DFS(i);
    }
}
// BFS
void BFS(int start) {
    bool visitedBFS[100] = {false};
    queue<int> q;
    visitedBFS[start] = true;
    q.push(start);
    while (!q.empty()) {
        int v = q.front();
        q.pop();
        cout << v << " ";
        for (int i = 0; i < n; i++) {
            if (graph[v][i] == 1 && !visitedBFS[i]) {
                visitedBFS[i] = true;
                q.push(i);
            }
        }
    }
}
```

```
int main() {
    int edges, u, v, start;
    cout << "Enter number of vertices: ";
    cin >> n;
    cout << "Enter number of edges: ";
    cin >> edges;
    // Initialize graph
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            graph[i][j] = 0;
    cout << "Enter edges (u v):\n";
    for (int i = 0; i < edges; i++) {
        cin >> u >> v;
        graph[u][v] = 1;
        graph[v][u] = 1;   // Undirected graph
    }
    cout << "Enter starting vertex: ";
    cin >> start;
    cout << "\nDFS Traversal: ";
    for (int i = 0; i < n; i++)
        visited[i] = false;
    DFS(start);
    cout << "\nBFS Traversal: ";
    BFS(start);
```

```
    return 0;
}
```

```
vvvlsravya@Vs-MacBook-Air-377 Practical_1-10 % g++ practical8_graph.cpp -o practical8 && ./practical8
Enter number of vertices: 8
Enter number of edges: 12
Enter edges (u v):
2,4
Enter starting vertex:
DFS Traversal: 1
BFS Traversal: 1 %
vvvlsravya@Vs-MacBook-Air-377 Practical_1-10 %
```

Result

Thus, the graph was implemented successfully and **DFS and BFS traversals** were performed.

PRACTICAL-9

Aim

To implement **Prim's algorithm** to find the **Minimum Spanning Tree (MST)** of a weighted graph.

Theory

Prim's algorithm is a **greedy algorithm** used to find the Minimum Spanning Tree of a connected, weighted and undirected graph.

It starts with any vertex and repeatedly selects the **minimum-weight edge** that connects a visited vertex to an unvisited vertex.

The MST contains all vertices with minimum possible total edge weight and has exactly **V − 1 edges**.

Where V = number of vertices.

Algorithm / Pseudocode

```
None

Prim(Graph, V)
    Set all key values = infinity
    Set key[0] = 0
    Set all vertices as not visited
    Repeat V times:
        u = vertex with minimum key value
            that is not visited
        Mark u as visited
        for each vertex v:
            if Graph[u][v] exists
```

```
        and v is not visited
        and Graph[u][v] < key[v]
            parent[v] = u
            key[v] = Graph[u][v]
    Print edges using parent[]
```

Program

```
#include <iostream>
#include <climits>
using namespace std;
int main() {
    int V;
    cout << "Enter number of vertices: ";
    cin >> V;
    int graph[100][100];
    cout << "Enter the adjacency matrix:\n";
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            cin >> graph[i][j];
        }
    }
    int parent[100];
    int key[100];
```

```
bool visited[100];
// Initialize
for (int i = 0; i < V; i++) {
    key[i] = INT_MAX;
    visited[i] = false;
}
key[0] = 0;
parent[0] = -1;
// Prim's Algorithm
for (int count = 0; count < V - 1; count++) {
    int minKey = INT_MAX;
    int u = -1;
    for (int v = 0; v < V; v++) {
        if (!visited[v] && key[v] < minKey) {
            minKey = key[v];
            u = v;
        }
    }
    visited[u] = true;
    for (int v = 0; v < V; v++) {
        if (graph[u][v] != 0 &&
            !visited[v] &&
            graph[u][v] < key[v]) {
            parent[v] = u;
```

```
            key[v] = graph[u][v];
        }
    }
}
int totalCost = 0;
cout << "\nEdges in Minimum Spanning Tree:\n";
for (int i = 1; i < V; i++) {
    cout << parent[i] << " - " << i
        << " : " << graph[i][parent[i]] << endl;
    totalCost += graph[i][parent[i]];
}
cout << "Total cost of MST = " << totalCost << endl;
return 0;
}
```

```
vvvlsravya@Vs-MacBook-Air-377 Practical_1-10 % g++ practical9_prim.cpp -o practical9 && ./practical9
Enter number of vertices: 5
Enter the adjacency matrix:
0 2 0 6 0
2 0 3 8 5
0 3 0 0 7
6 8 0 0 9
0 5 7 9 0

Edges in Minimum Spanning Tree:
0 - 1 : 2
1 - 2 : 3
0 - 3 : 6
1 - 4 : 5
Total cost of MST = 16
vvvlsravya@Vs-MacBook-Air-377 Practical_1-10 %
```

Result

Thus, **Prim's algorithm** was successfully implemented to find the **Minimum Spanning Tree** and its total cost.

PRACTICAL-10

Aim

To implement **Kruskal's algorithm** to find the **Minimum Spanning Tree (MST)** of a weighted graph.

Theory

Kruskal's algorithm is a **greedy algorithm** used to find the Minimum Spanning Tree of a connected, weighted and undirected graph.

It first sorts all edges in **increasing order of weight** and then selects the smallest edge that does not form a cycle.

The process continues until **V − 1 edges** are selected.

Algorithm / Pseudocode

```
None
Kruskal(Graph)
    Sort all edges in increasing order of weight
    Create separate set for each vertex
    MST = empty
    for each edge (u, v) in sorted edges
        if u and v belong to different sets
            Add edge (u, v) to MST
            Union the sets of u and v
        if MST contains V-1 edges
            stop
    Print MST
```

Program

```
#include <iostream>
#include <algorithm>
using namespace std;
struct Edge {
    int u, v, weight;
};
int parent[100];
// Find parent of a vertex
int findParent(int x) {
    if (parent[x] == x)
        return x;
    return parent[x] = findParent(parent[x]);
}
// Union two sets
void unionSet(int a, int b) {
    a = findParent(a);
    b = findParent(b);
    if (a != b)
        parent[b] = a;
}
int main() {
    int V, E;
```

```
cout << "Enter number of vertices: ";
cin >> V;
cout << "Enter number of edges: ";
cin >> E;
Edge edges[100];
cout << "Enter edges (u v weight):\n";
for (int i = 0; i < E; i++) {
    cin >> edges[i].u
        >> edges[i].v
        >> edges[i].weight;
}
// Sort edges by weight
sort(edges, edges + E, [](Edge a, Edge b) {
    return a.weight < b.weight;
});
// Initialize parent
for (int i = 0; i < V; i++)
    parent[i] = i;
int totalCost = 0;
int count = 0;
cout << "\nEdges in Minimum Spanning Tree:\n";
for (int i = 0; i < E && count < V - 1; i++) {
    int u = edges[i].u;
    int v = edges[i].v;
```

```
        if (findParent(u) != findParent(v)) {
            cout << u << " - " << v
                 << " : " << edges[i].weight << endl;
            totalCost += edges[i].weight;
            unionSet(u, v);
            count++;
        }
    }
    cout << "Total cost of MST = "
         << totalCost << endl;
    return 0;
}
```

```
vvvlsravya@Vs-MacBook-Air-377 Practical_1-10 % g++ practical10_kruskal.cpp -o practical10 && ./practical10
Enter number of vertices: 4
Enter number of edges: 5
Enter edges (u v weight):
1 2 10
1 3 6
1 4 5
2 4 15
3 4 4

Edges in Minimum Spanning Tree:
3 - 4 : 4
1 - 4 : 5
1 - 2 : 10
Total cost of MST = 19
vvvlsravya@Vs-MacBook-Air-377 Practical_1-10 %
```

Result

Thus, **Kruskal's algorithm** was successfully implemented to find the **Minimum Spanning Tree** and its total cost.

PRACTICAL-11

Aim

To implement the **Floyd-Warshall algorithm** to find the shortest paths between all pairs of vertices in a weighted graph.

Theory

The **Floyd-Warshall algorithm** is a dynamic programming algorithm used to find the **shortest distance between every pair of vertices** in a weighted graph.

It gradually considers each vertex as an intermediate vertex and updates the shortest distance between every pair.

Time Complexity

- **Time Complexity:** $O(V^3)$
- **Space Complexity:** $O(V^2)$

Where `V` = number of vertices.

Algorithm / Pseudocode

```
None

FloydWarshall(D, V)
   for k = 0 to V-1
      for i = 0 to V-1
         for j = 0 to V-1
            if D[i][k] + D[k][j] < D[i][j]
               D[i][j] = D[i][k] + D[k][j]
   Print D
```

Program

```
#include <iostream>
```

```
using namespace std;
#define INF 99999
int main() {
    int V;
    cout << "Enter number of vertices: ";
    cin >> V;
    int dist[100][100];
    cout << "Enter the adjacency matrix:\n";
    cout << "(Enter 99999 for no direct edge)\n";
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            cin >> dist[i][j];
        }
    }
    // Floyd-Warshall Algorithm
    for (int k = 0; k < V; k++) {
        for (int i = 0; i < V; i++) {
            for (int j = 0; j < V; j++) {
                if (dist[i][k] != INF &&
                    dist[k][j] != INF &&
                    dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] =
                        dist[i][k] + dist[k][j];
                }
```

```
            }
        }
    }
    cout << "\nShortest Distance Matrix:\n";
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            if (dist[i][j] == INF)
                cout << "INF ";
            else
                cout << dist[i][j] << " ";
        }
        cout << endl;
    }
    return 0;
}
```

```
vvvlsravya@Vs-MacBook-Air-377 Practical_1-10 % g++ practical11_floyd_warshall.cpp -o practical11 && ./practical11
Enter number of vertices: 4
Enter the adjacency matrix:
(Enter 99999 for no direct edge)
0 5 99999 10
99999 0 3 99999
99999 99999 0 1
99999 99999 99999 0

Shortest Distance Matrix:
0 5 8 9
INF 0 3 4
INF INF 0 1
INF INF INF 0
vvvlsravya@Vs-MacBook-Air-377 Practical_1-10 %
```

Result

Thus, the **Floyd-Warshall algorithm** was successfully implemented to find the **shortest paths between all pairs of vertices**.

PRACTICAL-12

Aim

To implement the **Travelling Salesman Problem (TSP)** and find the minimum-cost tour that visits every city exactly once and returns to the starting city.

Theory

The **Travelling Salesman Problem** finds the shortest possible route through a set of cities where each city is visited **exactly once**, and the tour returns to the starting city.

For a small number of cities, **backtracking** can be used to generate all possible tours and select the tour with minimum cost.

Time Complexity

- **Time Complexity:** O(n!)
- **Space Complexity:** O(n)

Where `n` = number of cities.

Algorithm / Pseudocode

```
None
TSP(current, count, cost)
    if count == n
        totalCost = cost + distance[current][0]
        if totalCost < minimum
            minimum = totalCost
        return
    for each city i
        if city i is not visited
            Mark i as visited
```

Name: V V V L Sravya (92460118112)

```
            TSP(i, count + 1,
                cost + distance[current][i])
            Unmark i
```

Program

```
#include <iostream>
#include <climits>
using namespace std;
int n;
int cost[20][20];
bool visited[20];
int minCost = INT_MAX;
void TSP(int current, int count, int totalCost) {
    // All cities visited
    if (count == n) {
        // Return to starting city
        int finalCost = totalCost + cost[current][0];
        if (finalCost < minCost)
            minCost = finalCost;
        return;
    }
    // Visit unvisited cities
```

```
    for (int i = 0; i < n; i++) {
        if (!visited[i]) {
            visited[i] = true;
            TSP(i,
                count + 1,
                totalCost + cost[current][i]);
            visited[i] = false;
        }
    }
}
int main() {
    cout << "Enter number of cities: ";
    cin >> n;
    cout << "Enter cost matrix:\n";
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cin >> cost[i][j];
        }
    }
    // Start from city 0
    visited[0] = true;
    TSP(0, 1, 0);
    cout << "\nMinimum travelling cost = "
         << minCost << endl;
```

```
    return 0;

}
```

```
vvvlsravya@Vs-MacBook-Air-377 Practical_1-10 % g++ practical12_tsp.cpp -o practical12 && ./practical12
Enter number of cities: 4
Enter cost matrix:
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0

Minimum travelling cost = 80
vvvlsravya@Vs-MacBook-Air-377 Practical_1-10 %
```

Result

Thus, the **Travelling Salesman Problem** was successfully implemented and the minimum travelling cost was obtained.